

Research paper

Kolmogorov-Arnold networks for algorithm design in battery energy storage system applications

Rolando Antonio Gilbert Zequera^{*}, Anton Rassõlkin, Toomas Vaimann, Ants Kallaste

Tallinn University of Technology, Department of Power Electrical Engineering and Mechatronics, Tallinn 19086, Estonia

ARTICLE INFO

Keywords:

Deep learning
Neural networks
Batteries
Artificial intelligence

ABSTRACT

Energy technologies and Artificial Intelligence (AI) are essential for the energy transition to a carbon-free future through decarbonization, digitalization, and decentralization. The Kolmogorov-Arnold Network (KAN) is a promising new type of Neural Network (NN) that can improve Deep Learning models and serve as an alternative to the Multilayer Perceptron (MLP) for complex tasks. This paper proposes using KANs to design algorithms for Battery Energy Storage System (BESS) applications, focusing on state estimation, Remaining Useful Lifetime (RUL), and charging management. A wide range of datasets is collected by performing extensive testing on battery cells to demonstrate the robustness of the algorithms, in addition to advanced techniques like cross-validation, Regularization, Bayesian optimization, and Fine-tuning to improve Model Performance Analysis. The resulting network architectures were designed using Keras and PyTorch APIs, stored in PyTorch state dictionaries and Hierarchical Data Format (HDF5) files, and tested on new battery datasets. The final KANs achieved over 96 % accuracy, outperforming Recurrent Neural Networks (RNNs), Convolutional Neural Networks (CNNs), and MLPs in algorithm design and BESS applications.

1. Introduction

In the accelerated landscape of the energy industry, the algorithm design of a Battery Energy Storage System (BESS) is essential to achieve energy transition. Artificial intelligence (AI) and energy integration highlight a remarkable development in beneficial applications such as diagnostics, charging management, predictive maintenance, state estimation, and Remaining Useful Lifetime (RUL). Different approaches are required to implement optimal technologies of energy systems, focusing on modeling and digitalization, with the electrochemical model, Equivalent Circuit Model (ECM), and Mathematical models being the most important to simulate, monitor, and predict the health and charge indicators of a BESS (Gilbert Zequera et al., 2023a, 2023d; Gilbert Zequera, 2022).

Several researchers and scholars have proposed not only Data Science techniques to deliver remarkable advances on several strategic topics such as digital twins, enterprise testing, and business consulting (Gilbert Zequera et al., 2023a, 2023b, 2023c) but also Deep Learning algorithms that explain the physics of a system and the behavior of Key Performance Indicators (KPIs) through AI-powered technology (Gilbert Zequera et al.), all to support the experience provided by energy

analysts.

In building and operating a BESS, the State of Charge (SOC) and State of Health (SOH) are critical for improving the performance of renewable technologies, making algorithm design essential for monitoring the efficiency of electrical machines and autonomous systems. Experimental measurements and battery testing track the charging and discharging, influenced by chemical structure, manufacturing characteristics, and actual usage, which are requested by user needs, industrial applications, and research goals. Accurate model evaluation through AI methods and scientific computing is crucial for addressing the challenges of the energy transition, ensuring effective performance monitoring, and supporting industrial procedures.

During the execution of AI methods in a BESS domain, there is an opportunity to optimize the architecture of a Neural Network (NN), thereby reducing the computational complexity in Deep Learning algorithms. However, when collecting different types of datasets and designing NNs, the challenge lies in explaining the physical and virtual entities of a BESS. This opacity regarding the internal mechanisms of the battery presents a significant obstacle when seeking to understand KPIs and user needs. As highlighted by Gilbert Zequera (R. A. Gilbert Zequera et.al), the search for the capability, robustness, and adaptability of AI

^{*} Corresponding author.

E-mail address: rogilb@taltech.ee (R.A. Gilbert Zequera).

methods in battery management is a challenge that unifies several branches of knowledge such as Computer Science, Energy Engineering, Mechatronics, and Software Engineering, which introduces the research question of designing an optimal algorithm whose mechanism can be implemented in both industry and academia beyond expectations.

Indeed, in such a context, developing, verifying, and implementing algorithm design from a Data Science and battery health perspective becomes highly applicable and valuable. Innovative and advanced NNs have been proposed as alternatives to solve complex Deep Learning problems such as Image Recognition and Computer Vision, Natural Language Processing, Speech Recognition and Voice Assistant, Recommended Systems, Autonomous Vehicles, etc (Du et al., 2016). In the literature, research studies have proposed incorporating AI methods for energy demand, and a remarkable contribution has been made through probabilistic models, Machine Learning, and Deep Learning approaches (Huang et al., 2020; Xiong and Zhou, 2023; Yang et al., 2023). In 2020, Yao et al. developed a novel method to achieve fault diagnosis in a BESS based on a Support Vector Machine (Yao et al., 2021). Later, in 2022, Li et al. developed a Deep Learning approach assisted by reinforcement learning to forecast an electric vehicle charging station (EVCS) (Li et al., 2023), and Jiang et al. implemented an isolated forest algorithm for voltage signal applications by detecting outliers (Chang et al., 2023).

The Kolmogorov-Arnold Network (KAN) is a promising AI algorithm based on the Kolmogorov-Arnold theorem, introduced by Liu et al. in 2024. Its significant applications have enabled scientists to rediscover physical and mathematical laws (Liu et al., 2024). Compared to traditional NNs, KAN has been shown to achieve faster neural scaling and offers notable improvements in Deep Learning models, enhancing both accuracy and interpretability (Liu et al., 2024; Bozorgasl and Chen, 2024).

This research addresses a critical gap in algorithm design for BESS applications by introducing and developing KANs, progressing from basic to advanced network architectures in the energy sector. The key innovation lies in moving away from conventional ECM designs and traditional NNs, instead leveraging Transfer Learning on battery operating data and comparing the performance of the resulting KANs with the most optimal categories of NNs. The study employs a comprehensive computing approach and Deep Learning methodology using PyTorch and Keras as high-level APIs. It rigorously tests the BESS under various conditions to assess charging management, Remaining Useful Lifetime (RUL), and state estimation across diverse datasets. By departing from previous research, this work not only fills a vital gap in battery development and AI technology but also offers a novel perspective on energy systems through the innovative application of KANs recently introduced by the scientific community.

The article is structured as follows: Section 1 provides a systematic review of recent advances in the algorithm design applied to a BESS and the motivation for this research. Section 2 gives a comprehensive review of the case studies with corresponding BESS applications. Section 3 presents an overview of the implemented NNs, highlighting the novelties of KANs, such as architecture, parameters, functionalities, and mechanisms. Section 4 elucidates the design of the optimized algorithms using a Deep Learning methodology and its corresponding procedures. The results and performance are explained in Section 5. Finally, conclusions drawn from the research are delivered in Section 6.

2. Case studies and applications

This section explains three different case studies to familiarize the reader with the relevant applications of a BESS, highlighting the level of complexity of algorithm design based on various user needs, battery properties, experimental tests, and operating conditions.

The case studies correspond to various datasets of a BESS subjected to experimental measurements through specific test criteria. Firstly, two Lithium-ion (Li-ion) cells are tested using a programmable DC electronic load to experimentally simulate the SOC by implementing a second-

order ECM. Secondly, two of the most recognized datasets, namely CALCE and NASA, are processed and analyzed to evaluate the RUL of prismatic and Li-ion battery cells. Finally, using the programmable DC electronic load from the initial case study, a series of battery tests are performed with several Li-ion cells of their corresponding modules to evaluate optimal charging management.

It is crucial to note that the added value of this section is to provide the high scope of algorithm design so that the optimal NN architecture can provide the most accurate performance and interpretability in multitasking areas for a BESS.

2.1. State estimation

A Mitsubishi i-MiEV battery pack manufactured by GS Yuasa, consisting of 88 LEV50N type Li-ion cells, was collected from ISEAUTO, an innovative Estonian autonomous electric vehicle (AEV) project located on the Tallinn University of Technology (TalTech) campus (Rassõlkin et al., 2018). Due to topics of strategic importance, energy cooperation, and research management, the battery pack was dismantled into several battery cell modules. For a comprehensive understanding of the LEV50N cells, Table 1 summarizes the cell parameters.

A total of ten battery tests for two Li-ion cells were conducted respectively to evaluate their charge indicators, all using a programmable DC electronic load through the UltraLoad Software for remote operation and monitoring. The programmable DC electronic load has an adjustable current rising speed from 0.001 A/μs to 5 A/μs, a list function that supports editing as many as 512 steps, dynamic mode up to 30 kHz, a readback resolution of 0.1 mV and 0.1 mA. Specifications of the battery tests consist of a resolution of 0.8 %, a slew rate of 0.001 A/μs, a frequency of 1 Hz, and a step duration of 1 second.

An ECM is a useful engineering approach that can simulate the operation of a BESS. The first-order ECM considers a parallel capacitor, voltage source, ohm resistance, and polarization resistance, while the second-order ECM includes not only the same elements but also implements a polarization resistance, which is divided into diffusion resistance and electrochemical resistance (Wu et al., 2010; Xia et al., 2017; Nemes et al., 2019).

In this research, a second-order ECM is coded using Python as the programming language. Furthermore, parameter estimation is achieved using the SciPy optimized library, focusing on root finding by Local (multivariate) optimization.

To visualize the results of the root findings in the battery tests, Fig. 1 shows a fitted curve and compares it with the experimental data.

To validate the results of the optimization and root finding algorithm in the parameter estimation for all battery tests, Table 2 provides the performance metrics consisting of Mean Absolute Error (MAE), Mean Square Error (MSE), and Root Mean Square Error (RMSE).

Finally, the best-fit parameters for each corresponding experimental measurement are collected and defined as input variables to run the second-order ECM, all to simulate the operation of the Li-ion cell and generate a new dataset.

For this research, the datasets corresponding to the state estimation application will be called “ECM dataset”, which contains SOC as a predicted variable, voltage, time, and current as input features, and which will be processed in the algorithm design for the following sections.

Table 1
LEV50N Battery cell parameters.

Parameter	Value	Unit
Nominal voltage	3.75	V
Operating voltage range	2.75–4.1	V
1-hr rate typical Capacity 25°C	50	Ah
Charge voltage limit at 25°C	4.1	V
Charge termination threshold current	0.5	A

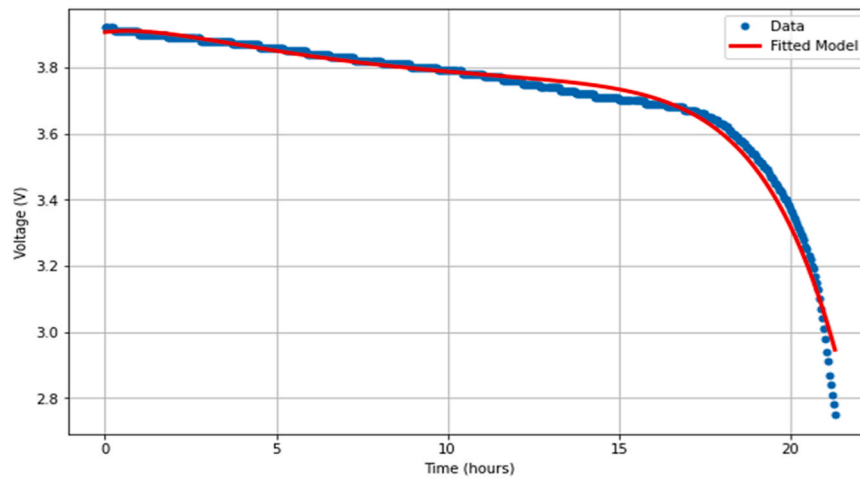


Fig. 1. Graphical predictions of the optimization algorithm to find the best-fit parameters.

Table 2

Validation results of the optimization and root finding algorithm.

Performance metrics	Results [%]
MAE	1.7038
MSE	0.0743
RMSE	2.7263

2.2. Remaining useful lifetime (RUL)

In this application, two of the most recognized datasets focused on RUL and End-of-Life (EOL) criteria are collected, which are NASA and CALCE. The EOL criteria is defined as a concept in a BESS that accounts for failure in performance or functionality, which is typically associated with 70–80 % of the full rated capacity (Arrinda et al., 2021).

The CALCE dataset, available from the Center for Advanced Life Cycle Engineering (CALCE), a research center at the University of Maryland, contains four prismatic cells cycled at a constant current of 1°C (He et al., 2011; Xing et al., 2013). The cells were tested on a charge profile that was a standard constant current/constant voltage protocol with a constant current rate of 0.5 C until the voltage reached 4.2 V and then held at 4.2 V until the charge current dropped below 0.05 A. The discharge cut-off voltage for these batteries was 2.7 V, and the total cell capacity was 1.1 Ah (Williard et al., 2020). In this dataset, the State of Health (SOH) represents the dependent variable, and the features used for the predictions are time, resistance, current, voltage, charge capacity, and discharge capacity (Gilbert Zequera et al., 2023c; Williard et al., 2020).

The NASA dataset, available from the NASA Ames Research Center (Macintosh, 2024), contains the record of four commercial 18650 Li-ion batteries, with each battery repeating three operations: charge, discharge, and impedance measurements. In this specific case study, only the discharging process is considered to evaluate the RUL. Initially, charging was performed in constant current (CC) mode at 1.5 A until the battery voltage reached 4.2 V and then continued in constant voltage (CV) mode until the charging current dropped to 20 mA. Subsequently, discharge was performed at a constant current level of 2 A until the battery voltage dropped to 2.7 V and experiments were stopped if the cells reached 70 % of the end-of-life criteria (Saha and Goebel, 2008a, 2008b). The predicted or dependent variable is the SOH, however, compared to the CALCE dataset, temperature indicates an additional independent variable (Gilbert Zequera et al., 2023b; Saha and Goebel, 2008b).

2.3. Charging management

Like the experimental measurements performed in the state estimation at the beginning of this section, this dataset corresponds to LEV50N cells, however, in this case study, sixteen cells from their respective modules are considered, conducting a total of 89 battery tests.

In this specific application, the charging management of Li-ion cells is evaluated using a programmable DC electronic load under several operating conditions, which are: 1) C rate, 2) operating voltage range, and 3) cut-off voltage range (R. A. Gilbert Zequera et.al). It is important to note that the operating conditions of the LEV50N cells were exhaustively measured in various scenarios through the UltraLoad Software for remote operation and monitoring.

Regarding data acquisition, the independent variables are current, time, voltage, energy, resistance, and discharge capacity. Additional features based on the input variables such as SOC and power are obtained using the Coulomb counting method; similarly, open circuit voltage (OCV) is calculated using the ECM approach and is defined as the predicted or target variable in the network architecture (R. A. Gilbert Zequera et.al; Saha and Goebel, 2008b).

For the purposes of this research, the dataset referring to the charging management application will be called “CHRG dataset”, which, like the previous ones, will be processed to achieve the optimal algorithm design in the next sections.

3. Artificial intelligence methods and neural networks

This section presents a brief review of the different AI methods focused on NNs, their corresponding mechanism, architecture, and functionalities, highlighting the implemented algorithms: 1) Kolmogorov-Arnold Network (KAN), 2) Multilayer Perceptron (MLP), and 3) Recurrent Neural Network (RNN) and Convolutional Neural Network (CNN). The workflow of each NN is illustrated in Fig. 2.

This diagram outlines the sequential structure for discussing the proposed NNs, emphasizing a systematic review of their definition, network architecture, mechanism, functionalities, and implementation. Each stage is shortly explained as follows:

- The definition explains to the reader the fundamental idea of each NN and its role in solving computational problems.
- The structural aspects of the corresponding NNs, such as layers, nodes, hyperparameters, and connections, are highlighted by different types of architectures.
- In the mechanism, the key content is based on how each NN works, breaking down the processes that allow them to learn and make predictions.

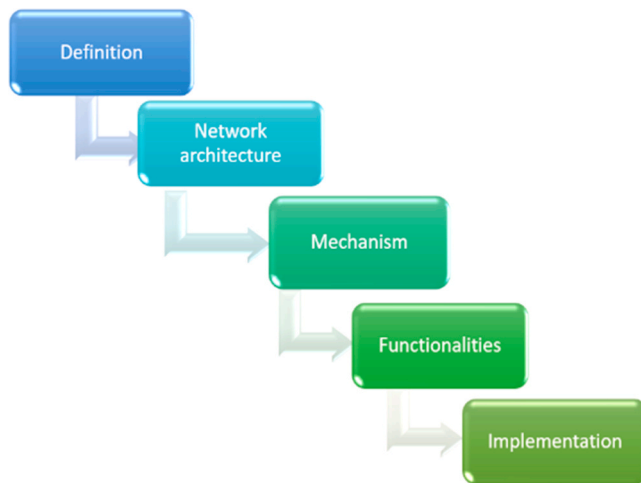


Fig. 2. Sequential structure diagram of the proposed NNs.

- Functionality is described by the strengths and capabilities of NNs through their network mechanisms and architectures in Deep Learning.
- The selected NNs for implementation are discussed through the practical aspects of the algorithm design and coding.

The main objective of the following subsections is to provide significant literature of implemented NNs before initializing the algorithm design, especially to understand the novel and innovative KAN.

3.1. Kolmogorov-Arnold network (KAN)

KANs are computational models inspired by the Kolmogorov-Arnold representation theorem, whose implications in the field of Deep Learning are offering new alternatives for building NNs. Initially, KANs are considered a promising alternative to MLP (Liu et al., 2024; Bozorgasl and Chen, 2024). However, due to their high level of accuracy and interpretability, the opportunity to compete with other categories of NNs, such as CNNs, RNNs, and ANNs in several fields of AI is encouraged.

Compared to MLPs, which have fixed activation functions on nodes or neural units, a KAN has learnable activation functions on numerical values, also known as weights, which are associated with the connections between neurons or nodes across different layers of the network. Furthermore, in the KAN mechanism a univariate function parameterized as a spline has the task of replacing each weight parameter, consequently allowing the switch between coarse-grained and a fine-grained grid (Liu et al., 2024). From a mathematical perspective, function composition also provides a powerful tool to explain the mechanism of a KAN by decomposing the multivariate function into univariate functions, this decomposition being a different approach than a traditional NN (Tian et al., 2023; Shayan Aziznejad and Unser, 2019).

Basis functions are defined as coefficients of the building blocks to create more complex functions and play an important role in the network architecture (Lee et al., 1999), so as an added value, a KAN can learn these basis functions at the edges, resulting in highly flexible and interpretive activation functions at each connection. The remarkable functionalities of KAN are also manifested by replacing all weight parameters by the coefficients within the edge activation function, thereby eliminating traditional linear weights from the network (Vaca-Rubio et al., 2024; Xu et al., 2024).

Regarding the main hyperparameters of a KAN, the width, grid, and spline order “k” integrates the network architecture. The width refers to the number of basis functions used to build the activation functions within each layer, the grid defines the level of detail at which the

interval over the activation function operates and captures in the network, finally, the “k” determines the degree of smoothness to parameterize the activation functions (Liu et al., 2024; Vaca-Rubio et al., 2024; Xu et al., 2024).

A summary of the KAN working framework is explained in the following points (Liu et al., 2024; Bozorgasl and Chen, 2024; Vaca-Rubio et al., 2024; Xu et al., 2024):

- **Input layer and processing.** At initialization, the input layer of a KAN extracts the multivariate input to prepare it for processing through the corresponding hidden layers. After that, each input variable is individually transformed by a set of univariate functions.
- **Hidden layers and univariate functions:** The hidden layers of the KAN architecture extract the univariate functions from the initial step. To complete this step, a sum of univariate functions is obtained, which represent a combination of transformed input variables.
- **Output layer:** The output layer extracts the sum of univariate functions from the hidden layer. Function composition is applied to the combination of transformed input variables, the results are calculated and then summed up to obtain the KAN predictions.

The innovation of the KAN architecture relies heavily on applying univariate functions to each input variable individually and then to the summed outputs, thereby reducing the complexity of the multivariate function by decomposing it into sums of simpler univariate functions. The parallel computation of multiple univariate transformations for each input variable is a major advantage compared to other categories of NNs, making it very efficient for certain types of problems in the Deep Learning field.

3.2. Multilayer perceptron (MLP)

A MLP is a type of artificial neural network (ANN) that consists of multiple layers of neurons. MLP architecture is characterized by the implementation of non-linear activation functions, which makes the network learn complex patterns in the data. A notable functionality of MLPs is their remarkable application in machine learning, as by learning non-linear relationships in the data, it turns them into powerful models for tasks such as regression, classification, and pattern recognition (R. A. Gilbert Zequera et al.; Heidari et al., 2020).

The MLP architecture comprises three layers: the input layer, the hidden layer, and the output layer. First, the input layer is the initial layer of the network, which processes the independent variables in the form of numbers. Second, is the hidden layer, which processes the information received from the input layer. Third, the output layer produces the results of the calculations applied to the network data (R. A. Gilbert Zequera et al.).

The mechanism of MLP is summarized in the next steps (Gilbert Zequera et al., 2023c; Heidari et al., 2020; Wilson and Tufts, 1994; Popescu et al., 2009):

- The activation rate of the hidden nodes is found using the inputs and the links from the input to the hidden layer. Each neuron in the hidden layer is connected to the neurons in the next layer.
- The corresponding weights of the neurons are updated with the help of the learning phase. The learning phase is repeated continuously until the error value exceeds the threshold level.
- Finally, the data is passed in a forward path from the input layer to the output layer, being the equivalent of a feed-forward that uses backpropagation to train all the nodes.

3.3. Recurrent Neural Network (RNN) and Convolutional Neural Network (CNN)

RNNs have an architecture based on recurrent connections and can model sequential data for sequence recognition and prediction. The

mechanism of an RNN is given by high-dimensional hidden states with nonlinear dynamics, emphasizing that the state of the hidden layer at a given time is conditioned to its previous state (Salehinejad et al., 2017).

Gated Recurrent Unit (GRU) and Long Short-Term Memory (LSTM) are types of RNNs designed to handle sequential data.

The LSTM is composed of three gates (forget, input, and output), whose beneficial mechanism allows RNNs to learn over many more time steps and control the flow of information to hidden neurons over long periods of time, thereby using more consistent errors. The corresponding gates of the LSTM retain the features extracted from previous time steps, regulating not only the amount of data entering at each corresponding time step, but also the number of weights being optimized (R. A. Gilbert Zequera et al.; Schmidt, 2019).

GRU has control units that modulate the flow of information within the unit, but without having separate memory cells. Unlike LSTM, GRU exposes the entire state at each step and computes a linear sum between the existing state and the newly computed state. Hidden layers containing memory cells cover the main functions of GRU networks. Cell state changes and maintenance depend on two gates in the cell: a reset gate and an update gate (R. A. Gilbert Zequera et al.; Salehinejad et al., 2017).

CNN is a feed forward NN that applies convolutional operations to the input instead of general matrix multiplication, being able to extract features from the data with convolutional structures in at least one of its layers (SAZLI, 2006). Compared to other types of NNs, a CNN has different layers that make up its intrinsic architecture, whose main functionalities are to control overfitting in the model, introduce nonlinearity, perform normalization, and combine features to make accurate final predictions (Li et al., 2022; Song et al., 2021).

The RNNs implemented in the algorithm design are GRU and LSTM, while a CNN-1D for CNN, whose performance will be compared and analyzed in the following sections.

4. Algorithm design

In this section, the algorithm design will be described, illustrated, and executed to evaluate the Model Performance Analysis subsequently. A Deep Learning methodology proposed by Gilbert Zequera et al (R. A. Gilbert Zequera et al.) in 2024 is implemented as an added value to strengthen robustness. However, compared to previous research, in this specific case, not only charging management applications but also RUL and state estimation are considered. In addition, both Keras and PyTorch are defined as APIs to demonstrate a high level of adaptability and effectiveness.

As for specific and advanced computing techniques, in the Training process, an initial network architecture is designed to consider input features of the corresponding datasets and NN hyperparameters. Fine-tuning is then performed using Bayesian optimization to achieve minimal validation loss, and then the optimal hyperparameters of the networks are collected to execute cross-validation and calculate performance metrics, avoiding later problems such as overfitting and underfitting. A summary of the algorithm design is explained in the following points:

- Data acquisition is completed by considering the corresponding datasets for each BESS application and separating the training, validation, and testing sets.
- Exploratory data analysis (EDA), Feature Engineering, and Feature Selection are executed to complete the Data processing. In this step, data distribution, correlation matrix, and Variance Inflation Factor (VIF) are useful tools to achieve a solid understanding of the problem depending on the type of application and identifying KPIs (R. A. Gilbert Zequera et al.; Tay, 2017).
- The initial network architecture is designed using the predictors and target variables from data processing, and the hyperparameters of the NN are selected. The training and validation sets are processed.

- Training is started and the loss is calculated on the validation set to obtain the initial performance of the NN. Fine-tuning is performed to achieve the minimum validation loss.
- Once the minimal validation loss is achieved, Fine-tuning is completed and optimal hyperparameters are collected. Cross-validation is performed to evaluate performance metrics in the validation step.
- PyTorch state dictionary (state_dict) and Hierarchical Data Format (HDF5) files are generated to store the resulting NNs. The testing set is processed, and final performance metrics are calculated in the Model evaluation step.

To conclude this introductory part, it is essential to mention that the interpretability of the algorithm design is given by both entities, the physical one, which refers to the operation of the BESS, and the virtual one, which is associated with the functionalities, architecture, and hyperparameters of the different NNs. This framework reflects and allows the user to understand the underlying factors that drive the predictions of the target variable in each application of the BESS.

4.1. Neural network architecture and training

After the successful Data processing completion, the training and validation sets were stored and consistently transformed into tensors to ensure alignment with time steps, features, and samples for algorithm design using Keras and PyTorch APIs.

Activation functions, metrics, and kernel regularizers are the main arguments that integrate the programming interface in the network architecture, so different matrices are created with a set of hyperparameters to monitor the Training and Validation steps. For RNNs, CNNs, and MLPs, batch size, learning rate, epochs, weight decay, and gamma regularizer are the selected hyperparameters, while for KANs, width, k-spline order, and grid are additional hyperparameters to optimize.

The selection of the above hyperparameters is intrinsically related to the impact on the performance and computational efficiency of the Training process, which affects the quantity of allocated error with which the NN weights are updated, and the amount of information the resulting architecture can capture and its suitability. As for the activation function, ReLu is selected for its sparsity and for being beneficial in reducing the probability of gradient vanishing.

Regularization techniques are crucial to implement and avoid overfitting or underfitting during the validation process, all to achieve convergence in the learning curves, so dropout, weight decay, and gamma regularizer are included in the hyperparameter set of the network architecture. Weight decay, also known as L2 Regularization, is a technique applied to the weights of a neural network to minimize a loss function by implementing a penalty on the norm of the respective weights (Loshchilov and Hutter, 2017). Similarly, the gamma regularizer is defined as a multiplicative factor by which the learning rate decays at each epoch (Li and Arora, 2019).

In addition to the above methods, the Adam optimizer is implemented to minimize losses and weights, stabilize Training, and help NNs converge to optimal solutions, which, in parallel with an early stopping criterion and a learning rate schedule, integrates the final network architecture. Early stopping reduces the risk of overfitting and saves time and computational resources by simplifying the model and preserving the best weights, while a learning schedule is tasked with avoiding exceeding the minimum learning rate and Fine-tuning the model parameters.

The notable differences in the Training step are based on the network architecture of KANs, which differ substantially from RNNs, CNNs, and MLPs due to the k-spline order, grid, and width, so these hyperparameters will be optimized first before obtaining the resulting NNs and completing the Fine-tuning process for mutual hyperparameters.

4.2. Fine-tuning and Bayesian optimization

In the field of AI and model interpretability, the need to achieve Fine-tuning plays a crucial role in both Validation and Model Performance Analysis. One of the techniques to automate algorithm design is Neural Architecture Search (NAS), which helps to find the optimal architecture by searching over a large hyperparameter space.

In terms of programming and computer science, understanding the key differences between trainable parameters and hyperparameters is a fundamental task. Trainable parameters refer to the parameters learned by the algorithm during Training, such as the weights of the NN, while on the other hand, hyperparameters are set before starting the learning process and are not updated in the learning step, thus dictating the overall structure and behavior of the model.

Fine-tuning using various optimization methods is a monumental challenge, but essential to ensure that models are accurate, efficient, and adaptable. Different types of hyperparameter optimization techniques include Evolutionary Algorithms, Reinforcement Learning, Grid Search, Bayesian optimization, and Random Search (Turner et al., 2021). In this research, due to its high level of convergence, computational efficiency, and fast convergence, Bayesian optimization is selected in the algorithm design (R. A. Gilbert Zequera et al.; Turner et al., 2021; Qin et al., 2017).

Bayesian optimization assumes that a specific probability distribution underlies the performance and determines the next set of hyperparameters to be evaluated considering previously observed combinations. Two important concepts are considered in the implementation of Bayesian optimization: exploration and exploitation. Exploration refers to selecting a point in a region that currently has the best results, while exploitation means choosing the point with the highest uncertainty at each iteration (Wu et al., 2019).

In the context of NNs and BESSs, the main goal of Bayesian optimization is to minimize the loss during the Training and Validation steps, achieving convergence on the learning curves and modeling the objective function. To successfully achieve this complex goal, a model called Gaussian Process is built, which refers to the infinite-dimensional natural analogue of the multidimensional Gaussian, and this process is used to model the unknown objective function and provides a posterior distribution over the function values given the observed data.

In the algorithm design, an acquisition function called Expected Improvement is implemented to provide a balance between exploitation and exploration, which determines the next set of points to be evaluated in the search space and quantifies the potential convenience of sampling a particular point, all while considering both the predicted values of the surrogate model and the uncertainty associated with those predictions.

As mentioned earlier, the Fine-tuning process via Bayesian optimization is implemented using Keras and PyTorch APIs. However, to offer an innovative solution, several programming functions were developed from scratch using Python libraries designed for sequential model-based optimization. First, a fitness function is created to evaluate the performance of the NNs based on the selected hyperparameters, to minimize the loss. Next, a Gaussian process is constructed during the Training phase, starting with an initial set of hyperparameters to learn the performance metrics of each NN. The Bayesian optimization search begins with these initial values and subsequently focuses on promising regions identified in earlier steps. In the third step, the model is sampled to maximize Expected Improvement, and the validation loss is calculated for each search iteration. Finally, the output function stores the set of hyperparameters that correspond to the minimum validation loss, and performance metrics are computed.

As for the KAN hyperparameters, the width, the grid and the k-spline order are the main elements to be optimized. In the case of the width, it is composed of the input size, the hidden size, and the output size, being the number of features and a single target variable the input and output size respectively, so only hidden size, grid, and k-spline order will be included in the Bayesian optimization for this type of NN.

Before obtaining the optimal and mutual hyperparameters of

different NNs, Bayesian optimization is implemented to select the core hyperparameters of the KAN. In this specific procedure, a loop iterates through the diverse types of datasets based on case studies until convergence is achieved based on the minimum validation loss. The results show optimal values of 16, 5, and 3 for the hidden size, grid, and k-spline order respectively.

The Bayesian optimization graph is a powerful tool that helps visualize where the optimization algorithm is likely to sample next, providing a solid understanding of the trade-off between exploration and exploitation. The graph illustrates exploration by sampling in areas with high uncertainty, while exploitation is shown by sampling in areas expected to yield the best results.

The above explanations are visualized in Fig. 3 and Fig. 4, showing a Bayesian optimization graph that is composed of the Gaussian process plot, and the Expected Improvement plot, considering the batch size of a KAN as an example hyperparameter.

In the Gaussian process plot, the Python fitness function provides observations along the data distribution of the selected hyperparameter with its associated validation loss, which is given by the Gaussian model. The X-axis plots the selected hyperparameter, while the Y-axis indicates the validation loss; the red points illustrate the observations provided by the Gaussian process.

Considering the Expected Improvement plot, the main objective is to analyze the different high-uncertainty sampling regions and the best results that exploration and exploitation yield. The X-axis indicates the selected hyperparameter and the Y-axis represents the Expected Improvement, quantifying the best current known value that can be expected if the function is evaluated at the corresponding hyperparameter.

Analyzing the Gaussian process plot, exploitation is represented by low areas suggesting promising regions based on existing data and giving minimal validation loss, on the contrary, areas with high uncertainty are indicated by exploration indicating maximum validation loss values that could be targeted by the acquisition function for further sampling. As can be seen in the Expected Improvement plot, areas with great values indicate a high probability of improving the current hyperparameter, conversely, lower areas represent regions where the model is confident that little or no improvement will be achieved because these areas have already been well explored.

As for the optimal hyperparameters of each NN in Bayesian optimization, a search is performed until convergence on the minimum validation loss is achieved. Fig. 5 shows a plot illustrating the surrogate models in algorithm design.

At first, it can be observed that the NNs show a different level of validation loss that decreases with the amount of search, however, there is a similarity in the final that illustrates the effectiveness of Bayesian optimization. Due to their architecture and functionalities, GRU and LSTM show a similar trend, however, KAN also provides even lower validation loss than MLP and CNN, giving an idea about the promising

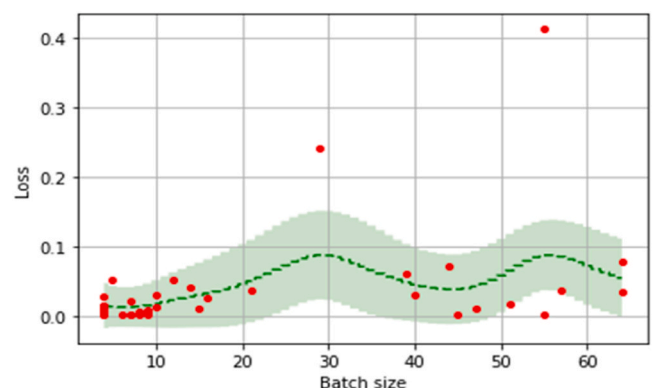


Fig. 3. Bayesian optimization graph, representing the Gaussian process.

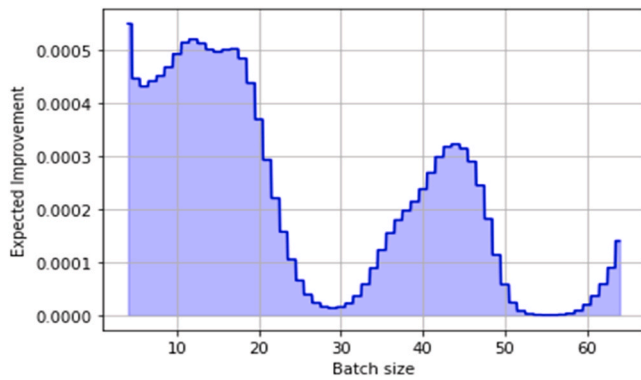


Fig. 4. Bayesian optimization graph, representing the Expected Improvement plot.

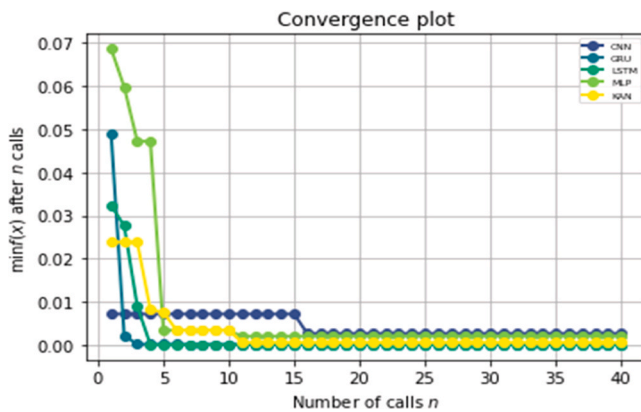


Fig. 5. Convergence plot showing surrogate models using Bayesian optimization. The X axis refers to the search number, the Y axis indicates the validation loss.

results that could be delivered in the next steps.

A graph of all combinations of hyperparameter values is shown in Fig. 6, exemplifying the Bayesian optimization of the CNN. The vertical axis illustrates the influence of a single dimension on a fitness function, called a “Partial Dependence plot” and the horizontal axis the hyperparameters. It visualizes the effects of changing one or more variables in the algorithm design and shows how the approximate fitness value changes with different values in that dimension. The yellow regions show areas where the loss on the validation set is lower, as opposed to the darker regions. The star in the graph represents the location where the optimal value of the hyperparameter is found.

It is fundamental to mention that Fine-tuning through Bayesian optimization was performed on all types of NNs explained in Section 3, so CNNs, MLPs, RNNs, and KANs highlight different sets of optimized

hyperparameters and performance metrics on the validation sets, which will be explored in the next subsection before providing the resulting NNs for each dataset in the Model Performance Analysis.

4.3. Validation

After Training and Fine-tuning are complete, the next step is to run cross-validation to measure the performance of each NN on the validation set. The goal of cross-validation is to provide an approximate performance of the model for data that will appear in the future. In addition, it is necessary to consider the importance of balancing underfitting and overfitting.

Underfitting refers to poor model performance on both the training and test sets; on the other hand, overfitting indicates that the model was over-tuned during Training, so it performs well on the training set but poorly on unseen being evaluated.

Due to the behavior of the training and validation losses at each epoch of the network architecture, overfitting, and underfitting can be identified from the learning curve. An underfitting plot shows high losses for both training and validation data at all epochs without significant improvement, hence it will be indicative of the lack of ability to learn the training set. On the other hand, in the case of overfitting, the training loss continues to decrease, while the validation loss starts to increase after reaching a minimum, thus the model fits the training data too closely, capturing noise along with the actual patterns.

In the programming framework, cross-validation is built in the objective function, in this case, the fitness function that was programmed in Python to run the Gaussian process. The objective function trains and evaluates the NN for each set of hyperparameters using cross-validation and returns the corresponding performance metrics. By using cross-validation in the objective function of Bayesian optimization, the selected hyperparameters are more likely to generalize well to unseen data, providing a more robust estimate of model performance and reducing the risk of overfitting across the search space.

A learning curve is a valuable tool for diagnosing the behavior of every NN in the Training and Validation steps. Key elements of an optimal fit consist of training losses that decrease to a plateau over epochs, while validation losses decrease, eventually plateau, and ideally remain close to the training loss. The numerical difference in the gap of a learning curve is the point at which the model’s validation loss is slightly larger than the training loss when both curves plateau. The specific size of the gap depends on the problem, but in general, a gap of 1 % is considered small and stable, indicating good generalization. Monitoring the gap of a learning curve during Training helps make decisions in the algorithm design about model complexity, Regularization, and when to stop Training to avoid overfitting

In Fig. 7, a good fit of the learning curve can be observed, showing a balance between bias and variance, indicating that the model generalizes well to new and unseen data. The curve illustrates the corresponding losses of a KAN, representing not only optimal tuning in Training and Validation but also convergence in a minimum number of

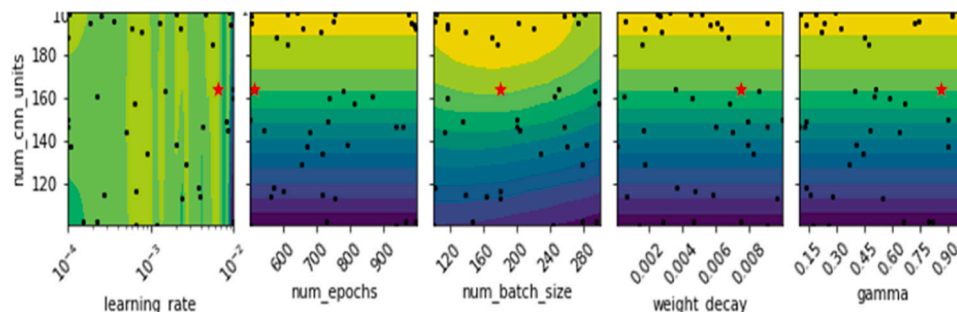


Fig. 6. Partial dependence plot of a Fine-tuning process using Bayesian optimization in a CNN.

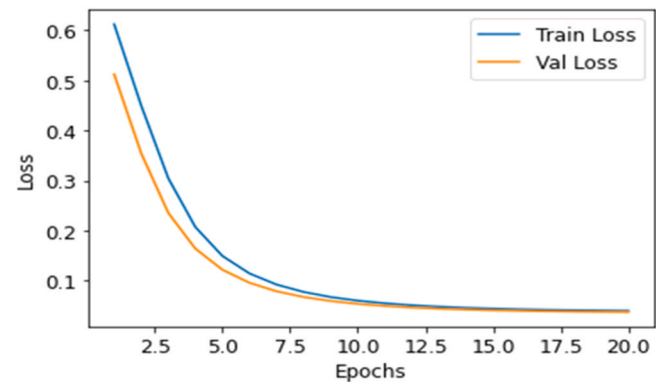


Fig. 7. Learning curve that shows an optimal fit in the Validation step.

epochs, thus reducing training time and computational complexity compared to the other NN categories.

Finally, the target variable is predicted for each dataset and performance metrics are obtained by calculating the MSE, MAE, and RMSE for the corresponding NNs. In addition, the Residual Sum of Squares (RSS) and Symmetric Mean Absolute Percentage Error (SMAPE) are the supported benchmarks employed to reflect the prediction’s accuracy and robustness to different scales of values. As mentioned in Section 3, GRU, LSTM, MLP, KAN, and CNN-1D are designed, implemented, and validated. Table 3, Table 4, Table 5, and Table 6 show the performance metrics in the Validation step.

From the performance metrics, it is crucial to evaluate the nature of the datasets and their main applications in the Model evaluation step. If the dataset contains outliers or noise that influence the model performance metric, it is advisable to select MAE due to the robust prediction tasks. On the contrary, if the dataset includes scenarios where large errors are particularly critical, the effect of MSE will cause these large errors to have a larger impact on the metric, which is useful when the goal of the model is to minimize such significant deviations. Regarding the RMSE, it is convenient to use it to balance the penalty for larger errors and whose interpretation of the error metric is in the same units as the target variable, so that the model predictions are accurate and interpretable.

As for the numerical values obtained in the validation results, the MAE is usually higher than the MSE, suggesting that the datasets have diminutive and uniformly distributed errors. This implies that the implemented NNs are generally accurate, without large deviations from the actual values, with operational variability being the core cause of error capture during battery testing. The implications of a larger MSE and RMSE than an MAE depend on the sensitivity of the model to outliers and greater errors, suggesting that predictions are heavily penalized by the MSE due to the quadrature effect, which could be beneficial information if large errors are particularly problematic in the BESS application.

The importance of the RSS relies on measuring the overall squared difference between the predicted and actual values, evaluating the error rate that the model accumulates over all data points, providing a holistic view of model accuracy, and handling continuous variables in the BESS operation. In the case of SMAPE, this benchmark normalizes errors relative to the actual and predicted values, making it robust to varying

Table 3
Validation results of the CALCE dataset.

NN	MSE [%]	MAE [%]	RMSE [%]	SMAPE [%]	RSS [%]
GRU	0.0136	0.5611	1.1663	0.9293	0.3143
LSTM	0.0132	0.5210	1.1529	0.8594	0.3072
MLP	0.4598	3.1563	6.7810	12.0454	2.1665
KAN	0.0127	0.4946	1.1306	0.7712	0.2953
CNN-1D	0.0652	1.2609	2.5554	2.4976	1.5077

Table 4
Validation results of the NASA dataset.

NN	MSE [%]	MAE [%]	RMSE [%]	SMAPE [%]	RSS [%]
GRU	0.0238	1.1564	1.5458	0.7113	0.6679
LSTM	0.0248	1.1471	1.5763	0.7672	0.6946
MLP	0.0728	2.1001	2.6996	1.3795	2.0373
KAN	0.0169	0.7900	1.3028	0.4995	0.4745
CNN-1D	0.0457	1.5921	2.1392	1.0833	1.2793

Table 5
Validation results of the ECM dataset.

NN	MSE [%]	MAE [%]	RMSE [%]	SMAPE [%]	RSS [%]
GRU	1.4947	0.8384	1.2226	0.1173	0.5655
LSTM	2.3191	0.9643	1.5228	0.1004	0.8774
MLP	0.6921	0.5337	0.8319	0.0704	0.2618
KAN	0.1240	0.8238	1.1138	0.1168	0.2122
CNN-1D	3.0969	1.3618	1.7598	0.1354	1.1717

Table 6
Validation results of the CHRG dataset.

NN	MSE [%]	MAE [%]	RMSE [%]	SMAPE [%]	RSS [%]
GRU	0.0014	0.2965	0.3758	0.4873	0.0759
LSTM	0.0017	0.3160	0.4191	0.4630	0.0944
MLP	0.0005	0.1673	0.2433	0.3665	0.0318
KAN	0.0010	0.1245	0.3163	0.0351	0.0314
CNN-1D	0.0012	0.2443	0.3244	0.4940	0.0566

scales, therefore ensuring that the algorithm ensures that the algorithm does not disproportionately penalize small deviations for large values or overlook significant deviations for small values in the BESS performance predictions.

Considering the implementation of Deep Learning methodology, Regularization, cross-validation, and Fine-tuning through Bayesian optimization are the testimony of the improvement and effectiveness of the different categories of NNs. Among the most relevant benefits are the reduction of computational resources in terms of training time and knowledge transfer, global optimization due to the balance between exploration and exploitation, better generalization to ensure that hyperparameters are configured to maximize NN performance, and adaptability to a wide range of network architectures.

The validation results show that KANs provide a higher level of performance compared to CNNs and RNNs; however, there are also notable performance metrics in the case of MLPs for ECM and CHRG datasets. A more detailed analysis and explanation will be provided in Section 5.

5. Model performance analysis

This section presents the optimal hyperparameters after Fine-tuning to compare and analyze each NN approach on different datasets. In the Model evaluation stage, the final performance metric is calculated and the NNs are evaluated on different test sets. At the end of this section, a discussion and analysis of the results are carried out, mainly focusing on KANs in the framework of AI methods for relevant applications of BESS.

5.1. Optimal network hyperparameters

The Fine-tuning process was executed through Bayesian optimization, using the Expected Improvement as the acquisition function and finding a mutual balance between exploration and exploitation for each hyperparameter, after that, cross-validation is performed to obtain the optimal hyperparameters of the network.

Table 7, Table 8, Table 9, and Table 10 show the optimal set of

Table 7

Optimal hyperparameters of the CALCE dataset.

NN	Batch size	Epochs	Learning rate	Weight decay	Gamma	Units
GRU	159	545	0.0100	1e–5	0.9711	128
LSTM	137	900	0.0030	1e–5	0.8371	240
MLP	38	881	0.0036	0.0053	0.1221	52
KAN	62	270	0.0022	1e–5	0.9802	16
CNN–1D	141	914	0.0017	0.0063	0.6806	100

Table 8

Optimal hyperparameters of the NASA dataset.

NN	Batch size	Epochs	Learning rate	Weight decay	Gamma	Units
GRU	100	800	0.0100	1e–5	1.00	150
LSTM	109	800	0.0059	1e–5	0.6705	100
MLP	195	1000	0.0009	0.0056	0.2765	50
KAN	36	129	0.0003	0.0006	0.5794	16
CNN–1D	202	1000	0.0087	0.0070	0.1013	145

Table 9

Optimal hyperparameters of the ECM dataset.

NN	Batch size	Epochs	Learning rate	Weight decay	Gamma	Units
GRU	182	829	0.0001	0.0005	0.9976	244
LSTM	102	999	0.0001	0.0024	0.9886	186
MLP	300	573	0.0007	0.0062	0.4524	107
KAN	221	35	0.0027	0.0009	0.9789	16
CNN–1D	180	512	0.0064	0.0075	0.8678	164

Table 10

Optimal hyperparameters of the CHRg dataset.

NN	Batch size	Epochs	Learning rate	Weight Decay	Gamma	Units
GRU	145	581	0.0008	0.0001	0.7717	102
LSTM	255	987	0.0013	0.0013	0.7780	216
MLP	240	500	0.0001	0.0010	0.9010	64
KAN	47	92	0.0002	1.27e–5	0.9985	16
CNN–1D	277	675	0.0003	0.0048	0.8742	104

hyperparameters of each NN for the corresponding datasets, after the successful completion of the Validation step.

In the context of Bayesian optimization convergence, the optimal hyperparameters are similar for the NN categories, however, the main differences depend on the applications of a BESS.

Regarding the RUL case study on the CALCE and NASA datasets, the learning rate and gamma of a KAN reach optimal values in a similar range as other types of NN, leading to a smooth and constant decrease in loss, however, significant differences are noted in the case of batch size and the most notable is found for the number of epochs and units, the latter being twice as small as the other NNs. The above result is explained based on the training time and knowledge transfer that the KAN manifests in the algorithm design due to its mathematical properties and functionalities, consequently impacting the dynamics of the network architecture, including training speed, memory usage, and faster convergence behavior.

In the case of state estimation and the ECM dataset, the complexity of the target variable reflects the convergence of the hyperparameters in a longer training time, providing high values of the epochs and units for the RNNs, however, in the case of MLP and CNN, there is mutual similarity which is given by the weight decay and gamma in the Regularization method. Analyzing the optimal hyperparameters of KAN, the epochs, learning rate, and number of units differ significantly from the

other NN categories, so the algorithm design converges in a reasonable number of iterations without overshooting or oscillating and balancing training speed and stability.

Charging management for the CHRg dataset shows the most notable results in the case studies, mainly due to the large number of input features and correlation in the Data processing step. While the learning rate and weight decay are in a similar range, the rest of the hyperparameters differ considerably regarding convergence and parallel computation. The nature of the KAN architecture allows the network to approximate any multivariate continuous function, making it a powerful tool in function approximation; this behavior is explained by the transformation of the input variables through univariate functions, and the combination of these transformations into the final output using further univariate functions and summation.

From the BESS perspective, selecting a C-rate and a cut-off voltage range are crucial tasks that will determine how long it takes for the BESS to charge or discharge in the experimental test, consequently they have a direct impact on the Training and Fine-Tuning in the case studies, so the batch size number and the learning rate are the two core hyperparameters that will intrinsically impact the different applications, all due to the number of samples used in a forward and backward propagation through the network architecture. Similarly, the nature of the BESS dataset depends on the input features and their relationship with the target variable, so hyperparameters such as weight decay, gamma regularizer, epochs, and number of units play a critical role in the algorithm design to determine the complexity of predictions on each application, achieving balanced learning, effective convergence, and generalization in the Training and Validation steps.

After completing the Fine-tuning and cross-validation steps, the optimal hyperparameter networks with their respective architectures were processed and saved using PyTorch and Keras APIs to create different hierarchical data formats (HDF5) and PyTorch state dictionary (state_dict) files. In the following subsection, the final performance metrics on the testing sets for each case study will be analyzed, compared, and discussed to highlight the outstanding performance of KANs and the novelties of this research.

6. Model evaluation

The corresponding testing sets are processed for each case study. In the RUL application, three different CS2 prismatic cells composed of LiCoO2 cathode integrate the CALCE dataset, while three commercial 18650 Li-ion batteries for the NASA dataset. Considering the state estimation, ten different ECM datasets are collected from an LEV50N cell integrating a Mitsubishi i-MiEV battery pack. In the case of the charging management application, a total of 89 battery tests were performed on sixteen LEV50N cells integrating four different battery modules.

To complete the Model evaluation step, all the different battery tests are stored in a Python list, processed, and transformed into a final set of matrices using Keras and PyTorch APIs to demonstrate the enormous level of adaptability and effectiveness in a programming environment. After that, the resulting network architectures were evaluated on the testing sets with the corresponding MSE, MAE, RMSE, SMAPE, and RSS to obtain the final performance metrics. Table 11, Table 12, Table 13, and Table 14 show a summary of the results in the Model evaluation step for each NN type and case study.

Table 11

Model evaluation results of the CALCE dataset.

NN	MSE [%]	MAE [%]	RMSE [%]	SMAPE [%]	RSS [%]
GRU	0.0220	0.7212	1.4818	1.2229	0.4464
LSTM	0.0193	0.6681	1.3894	0.9220	0.3934
MLP	0.0201	0.7240	1.4169	1.0602	0.4080
KAN	0.0189	0.6013	1.3721	0.8602	0.3841
CNN–1D	0.0232	0.9188	1.5204	1.3918	0.4688

Table 12

Model evaluation results of the NASA dataset.

NN	MSE [%]	MAE [%]	RMSE [%]	SMAPE [%]	RSS [%]
GRU	0.0592	1.6420	2.3756	0.9893	1.7291
LSTM	0.0457	1.4597	2.0737	0.9354	1.3680
MLP	0.0728	1.8399	2.6226	1.1708	2.0383
KAN	0.0410	1.0854	1.9540	0.6812	1.2191
CNN-1D	0.0549	1.8024	2.2723	1.1594	1.6117

Table 13

Model evaluation results of the ECM dataset.

NN	MSE [%]	MAE [%]	RMSE [%]	SMAPE [%]	RSS [%]
GRU	0.4302	4.4239	6.0624	5.9388	2.6165
LSTM	0.4156	4.4276	5.9194	6.0296	2.4790
MLP	0.2983	3.4999	4.8941	5.8712	2.3349
KAN	0.2934	3.0665	4.4073	5.5594	2.1173
CNN-1D	0.3089	3.6993	5.0661	5.7278	2.1102

Table 14

Model evaluation results of the CHRg dataset.

NN	MSE [%]	MAE [%]	RMSE [%]	SMAPE [%]	RSS [%]
GRU	0.1991	2.9660	3.7023	0.9385	4.7748
LSTM	0.3202	4.2372	4.9397	1.2505	9.0165
MLP	0.2405	3.9297	4.3954	1.3213	7.7094
KAN	0.0040	0.3051	0.3512	0.2273	0.0572
CNN-1D	0.1345	2.6333	3.0969	0.8931	3.4317

When analyzing the final performance metrics for each case study, NASA and CALCE datasets provide the highest level of accuracy in the Model evaluation, which is mainly due to the NNs' property of identifying sequential and non-sequential behavior leading to a decreasing trend as a function of SOH in the RUL. For charging management, the large number of features and different operating conditions increase the level of complexity in the prediction results for modeling the temporal dynamics of a BESS, thus the NNs' learning process takes considerable time to converge, and a higher error rate is obtained. The lowest rate of accurate predictions is calculated on the ECM datasets, all due to the high level of nonlinear behavior and complex objective functions, so that not only the convergence and training time increases but also the computation on charging and discharging, thus generating both forward and backward sequences in algorithm design.

Even under different operating conditions, scenarios, and BESS applications where battery testing was conducted, there is a tremendous accuracy of KAN outperforming the other NNs, with MSE less than 0.3 % and MAE and RMSE less than 4.5 % for all datasets. Considering the performance metrics at the Validation step, KANs still provide the minimum error rate in the same numerical range beyond the initial expectations in the algorithm design process. In contrast, MLP and CNN-1D have slightly higher accuracy in the Model evaluation but are still lower than KAN. At the same time, LSTM and GRU maintain remarkable performance without showing significant final improvement. Regarding additional metrics, the different network architectures perform accurately across all the case studies with less than 6.03 % and 9.10 % for SMAPE and RSS beneath high levels of non-linear behavior and health monitoring in a BESS, quantifying the overall error, and penalizing large deviations that ensure the algorithm learns effectively from the data.

The interpretation of KAN results is facilitated by its architecture, which simplifies high-dimensional problems by focusing on univariate functions. This approach is beneficial for modeling points where variable interactions are limited or can be modeled independently due to high correlations, such as the independent variables in a BESS. This reduces complexity and enhances performance in charging management applications.

For RUL prediction, the objective function is represented by a curve that gradually decreases as battery usage time increases. In this context, KANs are well-suited for additive models, where the output is the sum of individual contributions from each input, such as battery capacity across cycles.

In state estimation, prediction complexity increases significantly due to noisy inputs and non-linear relationships. However, KANs offer robustness by decomposing complex functions, isolating the influence of each input variable, and adapting to the various non-linearities in the data through univariate functions.

Fig. 8 and Fig. 9 graphically represent the RUL for the CALCE and NASA datasets respectively, indicating the number of cycles and the battery capacity. In the state estimation and ECM datasets, Fig. 10 illustrates the evolution of the SOC over a certain time. Finally, the OCV predictions with their corresponding SOC are visualized in Fig. 11. For visualization purposes, only the most accurate NNs are shown in the corresponding graphs.

Considering the RUL in Fig. 8 and Fig. 9, KAN's alignment with actual capacity curves shows its effectiveness in RUL prediction, leveraging its mathematical advantages across different datasets, and highlighting its capability of predicting the highly nonlinear and uncertain nature of battery degradation, thus offering a competitive option compared to other NNs. In Fig. 10, KAN predictions show a strong alignment with the actual SOC values across the entire period, especially during the intervals of both rapid transitions and stable phases of SOC, which is crucial for applications requiring real-time SOC estimation. Regarding Fig. 11, the KAN design allows it to adapt to static relationships like the OCV-SOC curve without relying on temporal or sequential dependencies, it maintains its accuracy across the entire SOC range, and superior performance ensures better reliability in SOC-OCV mapping, leading to more accurate and robust predictions in real-world applications.

To summarize this section, Fig. 12 illustrates a comparison of the different case studies, addressing the contributions of KANs in improving predictive accuracy and efficiency in BESS applications, and highlighting their expected impact on the final model performance. Finally, the following subsection will present a brief discussion that provides the novelties and challenges addressed by current research.

7. Discussion

In the initial steps, managing the entire data lifecycle is an essential task before initializing the algorithm design, so that understanding users' needs in the BESS application is achieved through EDA, Feature Engineering, Feature Selection, and Transformation. It was demonstrated that the predictors provide a level of interpretability depending on the case study, which helps to maximize and measure the predictive signals in further steps.

From the BESS domain, understanding the nature of the dataset allows the user to monitor the operational profile and physical interpretation of the BESS application, ensuring fairness and consistency in the selection of network hyperparameters. Regarding the architecture of the different KANs, a reciprocal relationship between hyperparameters was provided in a Fine-tuning process through Bayesian optimization for each case study, in which the learning rate and batch size monitor the behavior of predictions due to different step sizes and samples that update the weights of the network, and the number of epochs and units dictates the network dynamics based on training speed and convergence.

Gaussian process and Expected Improvement are added values of this research through Bayesian optimization, not only explaining the balance between exploration and exploitation for each hyperparameter in the network but also stochastically determining the optimal architectures in the Fine-tuning process, whose performance and combination values are visualized in Convergence Partial dependence plots.

Regarding the performance metrics in Model evaluation, all NNs

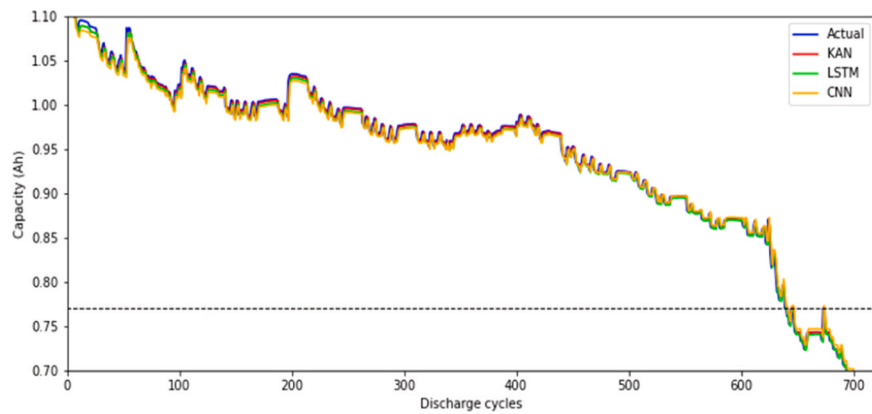


Fig. 8. Graphical predictions of the CALCE dataset. The X-axis indicates the number of discharged cycles, while the Y-axis represents the battery capacity. The horizontal line denotes the EOL criteria.

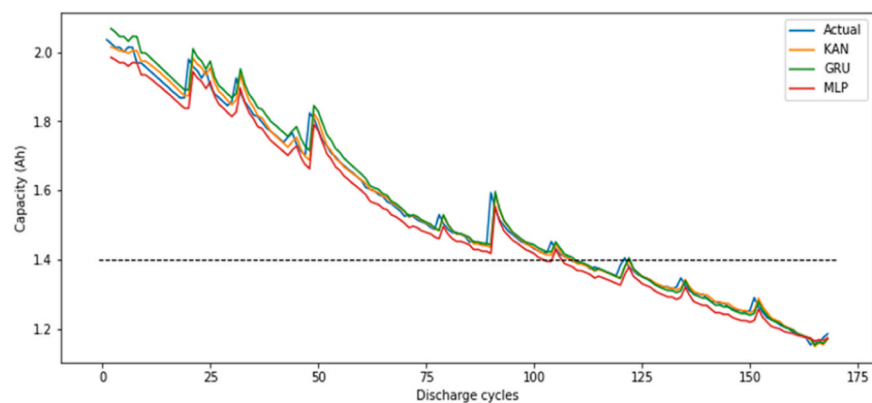


Fig. 9. Graphical predictions of the NASA dataset. The X-axis indicates the number of discharged cycles, while the Y-axis represents the battery capacity. The horizontal line denotes the EOL criteria.

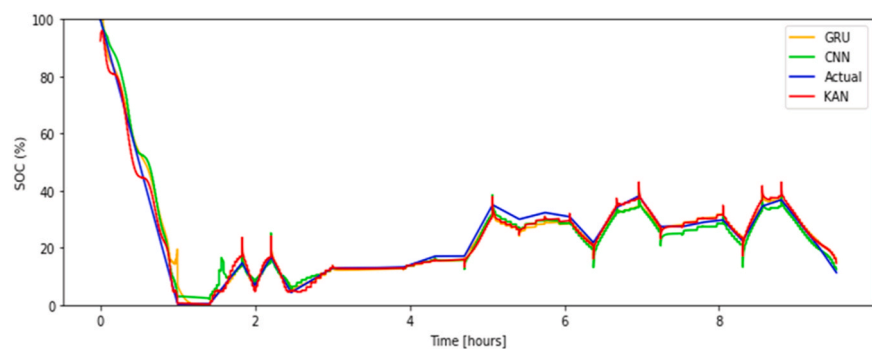


Fig. 10. Graphical predictions of the ECM dataset. The X-axis indicates the amount of time, while the Y-axis represents the evolution of the SOC.

provided an accuracy greater than 94 %, but the KANs showed the best results in all case studies with more than 96 %. The outstanding performance of KANs is attributed to their intrinsic property of accurate approximation of complex nonlinear functions and faster training convergence. Although the nature of the datasets differs due to the type of application, KANs demonstrated their beneficial mechanism in BESS tasks where: 1) Based on Health and Charge indicators, a high level of correlation between the independent variables is found and their relationship with the target variable is highly nonlinear, 2) The entire sequence contains both past and future time steps, which requires context from both directions to understand the operation of a BESS under different profiles, 3) An amount of high-dimensional data is collected containing noisy inputs, or limited data is processed during

specific charging scenarios.

Compared to existing Deep Learning models, Machine Learning methods, and physics-based approaches (Wei et al., 2022; Zhang and Li, 2022; Andersson et al., 2022), this research introduces the novel KANs in the energy framework, starting with algorithm design from a beginner level that initially familiarizes the reader with different case studies, until reaching an advanced network architecture that can make accurate predictions in the Model evaluation step. Furthermore, considering the BESSs applications, due to the high level of AI methods, NN categories, APIs, and programming tools, this research complements some proposed methodologies that not only focus on the aging state of a BESS through NNs (Cui et al., 2022a), but also in the diagnostics of RUL (Catelani et al., 2021; Qu et al., 2019), and state estimation using Kalman filtering,

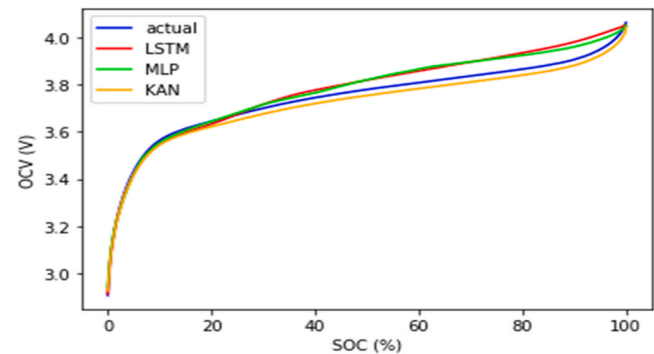


Fig. 11. Graphical predictions of the CHRg dataset. The X-axis represents the SOC percentage, while the Y-axis indicates the OCV.

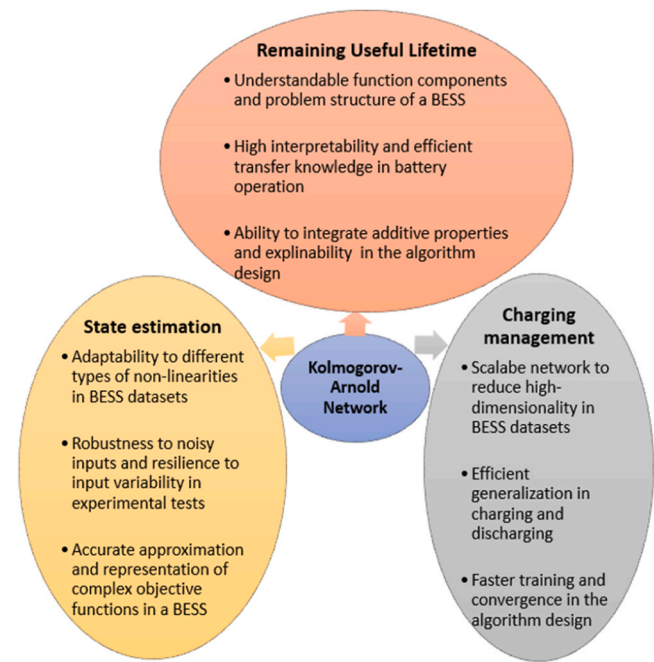


Fig. 12. Main contributions of the KANs in the Model evaluation for the different case studies.

NNs, and Transformer models (Cui et al., 2022b; Zeng et al., 2023; Zequera et al.).

In terms of accuracy, the proposed KANs improve existing methods for RUL, state estimation, and charging management, not only by comparing their resulting architectures with the most accurate NN

Table 15
Challenges addressed by the current research.

Topic	Challenges addressed
Battery technology	- Monitoring the performance of multiple battery cells with different chemicals properties, user needs, and BESS applications - Validation of experimental procedures for RUL, state estimation, and charging management - Accurate model evaluation based on AI methods and BESS operation
Algorithm design of a BESS	- Virtual entity represented by novel KANs in the energy domain focused on BESSs - High level of adaptability and AI interpretability based on APIs and programming tools - Innovative and promising neural architecture achieved through stochastic processes

categories such as CNN, RNN, and MLP, but also by explaining the design of these KAN architectures based on mathematical foundations, intrinsic properties, and computational functionalities. Regarding adaptability, the KANs provide the BESS framework to provide engineering behavior through Bayesian optimization in PyTorch and Keras APIs, whose future expectations can deploy the current methodology in a software environment through MLOps.

KAN's continuous multivariate function property represents a superposition of continuous univariate functions, which allows it to model highly nonlinear systems with fewer layers than traditional network architectures. KAN's uniqueness achieves better accuracy with a more compact structure, reducing computational overhead, facilitating more informed decision-making, and providing better adaptability to dynamic patterns over time. The KAN fills the technical gaps in this research not only by delivering competitive or superior performance with less than a 5.60 % error rate in all performance metrics for each case study, but also by addressing key challenges such as efficiency, interpretability, and adaptability to energy storage systems.

In the context of the future opportunities of a KAN within an energy framework, this article introduces the combination of AI methods to monitor the actual functioning of a BESS based on the user's needs, which vary significantly depending on the battery properties, available datasets, and experimental tests. Table 15 provides a summary of the challenges associated with predictive maintenance of a BESS addressed by this research.

The future study and improvement of KANs represent a crucial step toward the development, verification, and implementation of energy storage systems using AI methods, all to support beneficial tasks in both industry and academia, such as business consulting, research, and enterprise testing based on climate change mitigation.

8. Conclusion

In this study, the promising KANs were proposed, designed, compared to other NN categories, and employed to validate the experimental testing of different battery cells, marking a crucial step in the state estimation, charging management, and RUL applications of a BESS. This strategy was realized through the implementation of several computer science techniques that include Regularization, cross-validation, and Fine-tuning through Bayesian optimization to transform initial networks into several AI models, capable of emulating the intricacies of the BESSs. It was scientifically demonstrated that the design and execution of a KAN shows an optimal performance for battery development and AI-powered technology, answering the research question. Regarding the implications of the broader energy storage sector, this modest contribution leads to the initiative to promote ties of collaboration between different private administrations to establish the beginning of remarkable agreements in the domains of energy storage systems, sustainability, model serving, AI, and interpretability, model resource management techniques, and High-Performance Modeling.

From the quantitative perspective, the proposed KAN presents a unique and powerful alternative to traditional NNs for the algorithm design of a BESS, thus offering a balanced combination of simplicity, efficiency, and interpretability that engineers, researchers, and stakeholders can use to understand failure mechanisms, optimize performance, and make informed decisions about maintenance or replacement. Regarding the quantitative point of view, its compactness, robustness, and ability to generalize complex nonlinear systems make KAN particularly well-suited for predicting SOC, charging management, and RUL with less than 4.42 % in terms of MSE, MAE, and RMSE, while a maximum of 5.56 % and 2.12 % for SMAPE and RSS.

Among the most important novelties of this tremendous research are: 1) Proposing, validating, and evaluating KANs in the energy domain focused on a BESS, 2) Development of promising KAN architectures based on user needs, network hyperparameters, and battery chemistry, 3) Bayesian optimization that generates stochastic models for Health

and Charge indicators in a BESS, 4) Algorithm design to improve RUL, state estimation, and charging management applications using KANs and a Deep Learning methodology in an energy framework.

The outcome of this article is that the industrial and academic sectors have the most effective robustness to design virtual and physical entities of a BESS, whose mission is to repel any climate change problem that threatens sustainable stability, renewable sources, and energy integrity. In addition, the proposed research fulfills the strategic vision of the European Union to achieve the sustainable objectives of 2030, because specialists in the energy sector must respond effectively to any eventuality, which is our vocation as researchers, scientists, and scholars. Finally, it is of utmost importance to mention that this pioneering manuscript is considered a dynamic and strategic vision, whose efforts in the energy field are placing the domain of energy storage systems in a condition that exceeds expectations not only in the entire European territory but also around the world.

Author statement

We, the undersigned authors, affirm that all individuals listed have made substantial contributions to the research and approve of the manuscript's submission. We have no conflicts of interest to disclose.

CRediT authorship contribution statement

Rassõlkin Anton: Writing – review & editing, Supervision, Project administration, Funding acquisition. **Vaimann Toomas:** Supervision, Project administration. **Kallaste Ants:** Supervision, Project administration. **Gilbert Zequera Rolando Antonio:** Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The research has been supported by the Estonian Research Council under grant PRG2532 “Advanced Digital Tools to Accelerate the Development of Software-Defined Electric Vehicles under Grant PRG2532”.

Data availability

The data that has been used is confidential.

References

- Andersson, Malin, et al., 2022. Parametrization of physics-based battery models from input–output data: A review of methodology and current research. *J. Power Sources* 521, 230859.
- Arrinda, M., et al., 2021. Application dependent end-of-life threshold definition methodology for batteries in electric vehicles. *Batteries* 7, 1–20.
- Bozorgasl, Zavareh, Chen, Hao, 2024. Wav-kan: Wavelet kolmogorov-arnold networks. *arXiv Prepr. arXiv* 2405, 12832.
- Catelani, Marcantonio, et al., 2021. Remaining useful life estimation for prognostics of lithium-ion batteries based on recurrent neural network. *IEEE Trans. Instrum. Meas.* 70, 1–11.
- Chang, Chun, et al., 2023. Voltage fault diagnosis of a power battery based on wavelet time-frequency diagram. *Energy* 278, 127920.
- Cui, Zhenhua, et al., 2022b. Hybrid methods using neural network and Kalman filter for the state of charge estimation of lithium-ion battery. *Math. Probl. Eng.* 9616124.
- Cui, Zhenhua, et al., 2022a. A comprehensive review on the state of charge estimation for lithium-ion battery based on neural network. *Int. J. Energy Res.* 46, 5 5423–5440.
- X. Du, Y. Cai, S. Wang, and L. Zhang, Overview of deep learning, 2016 31st Youth Academic Annual Conference of Chinese Association of Automation (YAC), Wuhan, China, 2016, pp. 159–164, doi: 10.1109/YAC.2016.7804882.
- Gilbert Zequera, R., et al., 2022. Health and Charge Indicators for Battery Energy Storage Systems in Electric Vehicles Applications. 2022 IEEE 20th International Power Electronics and Motion Control Conference (PEMC). Brasov, Romania, pp. 427–432. <https://doi.org/10.1109/PEMC51159.2022.9962858>.
- Gilbert Zequera, R., et al., 2023a. Modeling battery energy storage systems based on remaining useful lifetime through regression algorithms and binary classifiers. *Appl. Sci.* 13, 7597. <https://doi.org/10.3390/app13137597>.
- Gilbert Zequera, R., et al., 2023b. Clustering and Outlier Analysis for Key Performance Indicators in Battery Energy Storage Systems applications. 2023 IEEE 17th International Conference on Compatibility, Power Electronics and Power Engineering (CPE-POWERENG): 17th IEEE International Conference on compatibility, Power Electronics and Power Engineering. Tallinn, Estonia. <https://doi.org/10.1109/CPE-POWERENG58103.20>.
- R. Gilbert Zequera et al., Data Science-based Techniques for Modelling and Diagnostics of Battery Cells Based on End-of-Life criteria, 2023 International Conference on Electrical Drives and Power Electronics (EDPE): 2023 International Conference on Electrical Drives and Power Electronics (EDPE). The High Tatras, Slovakia, September 25–27, 2023c. IEEE, 1–6. DOI: 10.1109/EDPE58625.2023.10274007.
- Gilbert Zequera, R., et al., 2023d. Overview of battery energy storage systems readiness for digital twins of electric vehicles. *IET Smart Grid* 6, 5–16. <https://doi.org/10.1049/stg2.1210>.
- R.A. Gilbert Zequera, V. Rjabtsikov, A. Rassõlkin, T. Vaimann and A. Kallaste, Deep Learning methodology for charging management applications in battery cells based on Neural Networks, in *IEEE Transactions on Intelligent Vehicles*, doi: 10.1109/TIV.2024.3417216.
- He, W., Williard, N., Osterman, M., Pecht, M., 2011. Prognostics of lithium-ion batteries based on Dempster–Shafer theory and the Bayesian Monte Carlo method, 10314 10321, Dec J. Power Sour. 196, 10314 10321, Dec.
- Heidari, Ali Asghar, et al., 2020. Ant lion optimizer: theory, literature review, and application in multi-layer perceptron neural networks. *Nat. -inspired Optim.: Theor., Lit. Rev. Appl.* 23–46.
- Huang, X., Wu, D., Boulet, B., 2020. Ensemble learning for charging load forecasting of electric vehicle charging stations. 2020 IEEE Electr. Power Energy Conf. (EPEC), Edmont., AB, Can. 1–5. <https://doi.org/10.1109/EPEC48502.2020.9319916>.
- Lee, Chien-Cheng, et al., 1999. Robust radial basis function neural networks. *IEEE Trans. Syst., Man, Cybern., Part B (Cybern.)* 29, 6 674–685.
- Li, Y., He, S., Li, Y., Ge, L., Lou, S., Zeng, Z., Jan. 2023. Probabilistic charging power forecast of EVCS: reinforcement learning assisted deep learning approach. *IEEE Trans. Intell. Veh.* 8 (1), 344–357. <https://doi.org/10.1109/TIV.2022.3168577>.
- Li, Z., Liu, F., Yang, W., Peng, S., Zhou, J., Dec. 2022. A survey of convolutional neural networks: analysis, applications, and prospects. *IEEE Trans. Neural Netw. Learn. Syst.* 33 (12), 6999–7019. <https://doi.org/10.1109/TNNLS.2021.3084827>.
- Li, Zhiyuan, Arora, Sanjeev, 2019. Expon. Learn. rate Sched. *Deep Learn. arXiv Prepr. arXiv* 1910, 07454.
- Liu, Ziming, et al., 2024. Kan: Kolmogorov-arnold networks. *arXiv Prepr. arXiv* 2404, 19756.
- Loshchilov, Ilya, Hutter, Frank, 2017. Decoupled weight decay regularization. *arXiv Prepr. arXiv* 1711, 05101.
- Macintosh, A. Li-Ion Battery Aging Datasets. Ames Research Center. 2010. Available online: <https://c3.ndc.nasa.gov/dashlink/members/38/> (accessed on 26 May 2024).
- Nemes, Raul Octavian, et al., 2019. Parameters identification using experimental measurements for equivalent circuit Lithium-Ion cell models. 2019 11th Int. Symp. . Adv. Top. Electr. Eng. (ATEE). IEEE.
- Popescu, Marius-Constantin, et al., 2009. Multilayer perceptron and neural networks. *WSEAS Trans. Circuits Syst.* 8, 7 579–588.
- Qin, Chao, Diego Klabjan, Russo, Daniel, 2017. Improving the expected improvement algorithm. *Adv. Neural Inf. Process. Syst.* 30.
- Qu, Jiantao, et al., 2019. A neural-network-based method for RUL prediction and SOH monitoring of lithium-ion battery. *IEEE Access* 7, 87178–87191.
- Rassõlkin, Anton, Raivo Sell, Mairo, Leier, 2018. Development case study of first Estonian Self-driving car ISEAUTO. *Sci. J. Riga. Tech. Univ. -Electr., Control Commun. Eng.* 14.1, 81–88.
- B. Saha and K. Goebel, Battery Data Set. Moffett Field, CA, USA: NASA Ames Prognostics Data Repository, NASA Ames Research Center, 2008a.
- B. Saha and K. Goebel, Uncertainty management for diagnostics and prognostics of batteries using Bayesian techniques, in *Proc. IEEE Aerosp. Conf.*, Mar. 2008b, pp. 1–8.
- H. Salehinejad et al., Recent advances in recurrent neural networks, 2017, *arXiv: 1801.01078*.
- SAZLI, M.H., 2006. A brief review of feed-forward neural networks. *Commun. Fac. Sci. Univ. Ank. Ser. A2-A3: Phys. Sci. Eng.* 50 (01). <https://doi.org/10.1501/commua1-2.0000000026>.
- R.M. Schmidt, Recurrent neural networks (rnns): A gentle introduction and overview, 2019, *arXiv:1912.05911*.
- Shayan Aziznejad, Unser, Michael, 2019. Deep spline networks with control of lipschitz regularity. *ICASSP 2019-2019 IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Pages 3242–3246. IEEE.
- Song, Jiancai, et al., 2021. Predicting hourly heating load in a district heating system based on a hybrid CNN-LSTM model. *Energy Build.* 243, 110998.
- Tay, R., 2017. Correlation, variance inflation and multicollinearity in regression model. *J. East. Asia Soc. Transp. Stud.* 12, 2006–2015.
- Tian, J., Chen, C., Shen, W., Sun, F., R.Xiong, 2023. Deep learning framework for lithium-ion battery state of charge estimation: Recent advances and future perspectives. *Energy Storage Mater.* 61.

- Turner, Ryan, et al., 2021. Bayesian optimization is superior to random search for machine learning hyperparameter tuning: Analysis of the black-box optimization challenge 2020. *NeurIPS 2020 Compét. Demonstr. Track*. PMLR.
- Vaca-Rubio, Cristian J., et al., 2024. Kolmogorov-arnold networks (kans) for time series analysis. *arXiv Prepr. arXiv* 2405, 08790.
- Wei, Zheng, Qiu He, Yan, Zhao, 2022. Machine learning for battery research. *J. Power Sources* 549, 232125.
- Williard, N., He, W., Osterman, M., Pecht, M., 2020. 'Comparative analysis of features for determining state of health in lithium-ion batteries, *Int. J. Prognostics. Health Manag.* 4 (1), 1–7.
- Wilson, E., Tufts, D.W., 1994. Multilayer perceptron design algorithm. *Proc. IEEE Workshop Neural Netw. Signal Process., Ermioni, Greece* 61–68. <https://doi.org/10.1109/NNSP.1994.366063>.
- Wu, Guoliang, Chunbo Zhu, Chan CC, 2010. Comparison of the first order and the second order equivalent circuit model applied in state of charge estimation for battery used in electric vehicles. *J. Asian Electr. Veh.* 8. 1 1357–1362.
- Wu, Jia, et al., 2019. Hyperparameter optimization for machine learning models based on Bayesian optimization. *J. Electron. Sci. Technol.* 17. 1 26–40.
- Xia, Bizhong, et al., 2017. A novel observer for lithium-ion battery state of charge estimation in electric vehicles based on a second-order equivalent circuit model. *Energ.* 10. 8 1150.
- Xing, Y., Ma, E.W.M., Tsui, K.-L., Pecht, M., 2013. An ensemble model for predicting the remaining useful performance of lithium-ion batteries. *Microelectron. Rel.* 53, 811–820.
- Xiong, X., Zhou, L., 2023. A method of short-term load forecasting at electric vehicle charging stations through combining multiple deep learning models. 2023 2nd Asia Power Electr. Technol. Conf. (APET), Shanghai, China 740–744. <https://doi.org/10.1109/APET59977.2023.10489548>.
- Xu, Kunpeng, Lifei Chen, Shengrui, Wang, 2024. Kolmogorov-Arnold networks for time series: bridging predictive power and interpretability. *arXiv Prepr. arXiv* 2406, 02496.
- Yang, Z., Hu, T., Zhu, J., Shang, W., Guo, Y., Foley, A., Jan. 2023. Hierarchical high-resolution load forecasting for electric vehicle charging: a deep learning approach. *IEEE J. Emerg. Sel. Top. Ind. Electron.* 4 (1), 118–127. <https://doi.org/10.1109/JESTIE.2022.3218257>.
- Yao, Lei, et al., 2021. An intelligent fault diagnosis method for lithium battery systems based on grid search support vector machine. *Energy* 214, 118866.
- Zeng, Yi, Yan, Li, Tong, Yang, 2023. State of charge estimation for lithium-ion battery based on unscented Kalman filter and long short-term memory neural network. *Batter.* 9. 7 358.
- R.A.G. Zequera, A. Rassölkin, T. Vaimann and A. Kallaste, Charge diagnostics and state estimation of Battery Energy Storage Systems through Transformer models, in *IEEE Access*, doi: 10.1109/ACCESS.2025.3532858.
- Zhang, Ying, Li, Yan-Fu, 2022. Prognostics and health management of lithium-ion battery using deep learning methods: a review. *Renew. Sustain. Energy Rev.* 161, 112282.